

WiFiGate — Technical Handbook

WiFiGate — Technical Handbook

WiFiGate Handbook — End-to-End Professional Guide

*A complete learning document for the WiFiGate system: how everything works, **why** it is built this way, which libraries are used, all the flows, and the important logs for each stage. Written by reading the actual source code, not from memory. If you want a general overview, read `README.md` first. This document is a deeper layer: a learning guide + reference. ¹ A Hebrew version of this handbook is available at `WIFIGATE_HANDBOOK_HE.md`.*

Table of Contents

1. Big Picture: what WiFiGate is and why
 2. The tech stack — every library and why it was chosen
 3. Code map — where everything lives
 4. Data model — Gate, Role, Group, Dual Control
 5. The storage layer — why local-first and how it works
 6. State management — the custom store
 7. The core end-to-end flows
 8. The secure BLE protocol `SECURE_V3` — in depth
 9. Invitation-link encryption — in depth
 10. Deep Linking — how a link becomes a screen
 11. Auto Open — proximity-based opening
 12. FOTA / OTA — firmware update
 13. Nearby Wi-Fi Setup — the local setup page
 14. Important logs — full catalog by tag
 15. Permissions and platform
 16. Secrets and build inputs
 17. Local development, checks, and release
 18. Security boundaries — what is protected and what is not
 19. Cheat Sheet — quick references
-

1. Big Picture

WiFiGate is a **React Native / Expo** app for controlling physical gates driven by WiFiGate firmware (based on the ESP32-C6). The app is essentially two things at once:

20. A **gate wallet** on the phone — a list of gates with different access levels and lifetimes.
21. A **secure control client** for the device — it opens a gate over Bluetooth Low Energy using an encrypted protocol.

The app's three jobs

The app stores gate metadata **locally** (local-first) and uses that list to drive three jobs:

- **Opening a gate over BLE** (the core).
- **Sharing / receiving access** via encrypted invitation links.
- **Entering local setup over Wi-Fi** (the <http://192.168.4.1/> page).

Guiding principle: separation of security layers

This is a critical point for understanding the whole architecture — there are **two separate security problems** with **two separate solutions**:

Layer	What it protects	Mechanism
BLE (SECURE_V3)	phone↔gate communication	ECDH + HKDF + AES-256-GCM + HMAC
Invitation links	the payload inside the URL	AES-256-GCM with a key derived from HMAC_URL_SECRET

They do **not** replace each other. An encrypted link does not protect BLE, and SECURE_V3 does not protect the URL. See Section 18.



2. The Tech Stack

Source: package.json. Below are the significant libraries and **why** they are there.

Core — Runtime and Navigation

Library	Role	Why
expo ~54 / react-native 0.81 / react 19	the base	Expo provides a dev-client, builds (EAS), and config plugins without touching native code by hand
expo-router ~6	file-based routing	every file in app/ is a route; deep-linking is built in
expo-dev-client	custom dev build	required — the app depends on native modules (BLE, Wi-Fi) that don't exist in Expo Go

<code>react-native-reanimated / gesture-handler / worklets</code>	animations and gestures	smooth UI, drag-to-reorder
<code>zustand</code>	state (available)	Note: the main gate store (<code>gatesStore.ts</code>) is hand-built and does not use zustand. See Section 6

Cryptography — the heart of security

Library	Role
<code>@noble/curves</code>	P-256 curve (ECDH for the handshake)
<code>@noble/hashes</code>	SHA-256, HKDF, HMAC
<code>@noble/ciphers</code>	AES-256-GCM
<code>react-native-get-random-values</code>	polyfill for <code>crypto.getRandomValues</code> (must load first — see line 1 of <code>app/_layout.tsx</code>)
<code>buffer / base-64</code>	base64 encoding for BLE and links

`@noble/*` were chosen because they are pure-JS crypto libraries, no native dependency, audited, and suitable for React Native (which lacks a full `crypto`).

BLE and Wi-Fi — the physical transport

Library	Role
<code>react-native-ble-plx</code>	the BLE core: scan, connect, GATT, notify/write
<code>react-native-bluetooth-state-manager</code>	Bluetooth state (on/off) and enable request
<code>react-native-wifi-reborn</code>	scan/connect to the gate's AP (the Nearby setup flow)
<code>react-native-webview</code>	displaying the local <code>192.168.4.1</code> setup page

Identity and Auth

Library	Role
<code>@react-native-firebase/auth + firebase</code>	SMS OTP phone authentication
<code>@react-native-google-signin/google-signin</code>	Google identity (fills email for BLE)
<code>expo-apple-authentication</code>	Sign in with Apple
<code>expo-auth-session</code>	OAuth flows
<code>libphonenumber-js + country-list</code>	normalizing/validating phone numbers and country codes

I/O and UI

`expo-camera` (QR scanning), `react-native-qrcode-svg` (QR generation for sharing), `expo-contacts` (picking an invitee from contacts), `expo-file-system + expo-sharing` (CSV / file export), `react-native-draggable-flatlist` (reordering gates), `lottie-react-native` (animations), `@react-native-async-storage/async-storage` (local storage).

3. Code Map

```
app/                                # Expo Router – each screen is a route
  _layout.tsx                       # root: deep-link routing, auth gating, splash,
auto-open boot
  index.tsx                         # home screen: gate list, groups, search, reorder
  scan.tsx                         # QR scanning + import
  gate/[id].tsx                    # link import + gate details screen
  open-gate/[id].tsx               # quick open (from widget/deep link)
  group/[id].tsx                  # group screen
  dual-control/[id].tsx           # dual-control screen
  nearby-wifigate-link.tsx        # entry to Nearby Wi-Fi setup
  wifigate-setup.tsx              # the WebView of the local setup page
  firmware-update.tsx            # OTA screen
  settings/[id].tsx               # gate settings
  auth/login.tsx                  # login (provider)
  auth/phone.tsx                  # phone OTP

src/
  models.ts                       # all types: Gate, UserRole, GateGroup,
DualControl...
  storage.ts                      # AsyncStorage – all keys, load/save,
normalization
  gatesStore.ts                   # the central state store (manual observable) +
merge rules
  gateService.ts                  # guest-access validation and service logic
  openGateFlow.ts                # the gate-open orchestration
(permissions→BT→BLE)
  gateOpenBlockReason.ts         # mapping error messages to block reasons
  gateSecurity.ts                # parsing/normalizing the security bundle
(SECURE_V3)

  ble/
    openGate.ts                  # the BLE core (scan/connect/handshake/frames) –
the big file
    secureProtocol.ts            # the SECURE_V3 crypto (handshake, keys, frames)
    bluetoothState.ts           # Bluetooth state + ensureBluetoothReady
    permissions.ts              # BLE permissions

    inviteLinkCrypto.ts          # encrypt/decrypt of link payloads (AES-GCM)
    linkService.ts              # parse/build invitation links
(single/group/dual)
    dualControlInviteImport.ts   # import of a dual-control invitation
    groupInviteImport.ts        # import of a group invitation

  auth/                           # LocalAuthContext, providerIdentity,
google/apple
  auto-open/                     # runtime + native bridges + headless task +
config
  fota/                          # publicFota (Firebase), driveFota (legacy)
  widgets/                       # gate widget (Android) + headless open
  wifi/                          # useGateWifiConnectionGuard
  crypto/                        # hmacSecrets (reconstructs HMAC_URL_SECRET)
  i18n/                          # languages and translations
  permissions/                  # appLaunch, contacts, wifi
  utils/                        # datetime, gateCommand, guestAccess,
subscription...
  components/                   # all UI: modals, gate-settings, admin-access...
  features/my-gates/            # the home-screen collections (tiles, helpers)

docs/
  ble-security-protocol-v3.md     # the formal SECURE_V3 spec
  WIFIGATE_HANDBOOK_EN.md        # this document
  WIFIGATE_HANDBOOK_HE.md        # Hebrew version

README.md                        # general overview
README_WIFIGATE_LINK_INVITATIONS.md # the full invitation-link protocol
```

```
README_OTA_FIREBASE.md      # OTA from Firebase Storage
WIFIGATE_API_README.md      # links issued by the WIFIGATE API
```

4. Data Model

Source: `src/models.ts`.

4.1 Gate

The central entity. Important fields:

```
type Gate = {
  id: string;           // local identifier (= systemId currently)
  systemId: string;     // hardware id, used in BLE and links. e.g.
  "9876543210123456"
  systemName: string;   // the original/imported name
  displayName: string;  // the name the user can edit
  role: UserRole;       // ★ the role decides everything (see 4.2)
  subscription?: "OFFLINE" | "ONLINE" | "WIFI" | "ALL_IN_ONE";
  security?: GateSecurityConfig; // the SECURE_V3 bundle
  gateCommand?: GateCommandType; // the command type the firmware reported
  (PULSE/TOGGLE/PUSH_BUTTON...)
  autoOpen?: GateAutoOpenSettings;
  // sharing/hosting:
  hostPhone/hostName/hostEmail? // the inviter
  guestPhone?                    // the invitee (in a private guest link)
  accessStart/accessEnd?         // validity window (ms)
  // registration for PRE_USER/PRE_ADMIN:
  residentName/floorNumber/apartmentNumber/parkingNumber?
  openHistory?: GateOpenHistoryEntry[]; // local log of recent opens (newest-
  first)
};
```

4.2 Role — the role decides everything

```
type UserRole = "ADMIN" | "USER" | "GUEST" | "PRE_USER" | "PRE_ADMIN";
```

Role	Meaning	Source	BLE command after import
ADMIN	confirmed admin	after firmware confirmation	OPEN / TOGGLE_* / AUTO_OPEN
USER	confirmed permanent user	after firmware confirmation	OPEN / TOGGLE_* / AUTO_OPEN
GUEST	temporary access with a time window	guest link	OPEN_GUEST / TOGGLE_*_GUEST
PRE_USER	invited as user, not yet confirmed	user link (<code>r=PU</code>)	ADD_USER
PRE_ADMIN	invited/added as admin, not yet confirmed	QR/manual or admin link (<code>r=PA</code>)	ADD_ADMIN

****Why `PRE_*`?** Because the app is local-first — it cannot know that the firmware confirmed you. Until the device confirms, you are in a "pending" state. When `ADD_USER`/`ADD_ADMIN` runs successfully against the gate, the firmware confirms, and the `onRoleUpgrade` callback upgrades `PRE_USER`→`USER` / `PRE_ADMIN`→`ADMIN` (see `maybeUpgradeRole` in `openGate.ts` and the `persist` in `openGateFlow.ts`).

4.3 Role merge rules (why a gate isn't "destroyed")

Source: `addGate` in `gatesStore.ts`. The system protects permanent access from an accidental downgrade:

- A permanent gate (ADMIN/USER) is **not** downgraded to GUEST when importing a guest link.
- A permanent role is **not** overwritten by a weaker `PRE_*` role.
- A guest gate **can** be upgraded to a permanent/pending role.
- A custom name (`displayName`) is preserved if a later import brings only a default name.

4.4 GateGroup and DualControl

- **GateGroup** — a purely local UI container. It changes no firmware/BLE behavior. It supports nesting via `parentGroupId`. It holds `gateIds[]` and `dualControlIds[]`.
 - **DualControl** — a pair of gates (`firstGateId/secondGateId`) with two action labels (`firstLabel/secondLabel`, e.g. "Open"/"Close"). Every dual-control invitation contains **exactly 2 gates**.
-

5. The Storage Layer

Source: `src/storage.ts`. The system is **local-first** — gate state lives in `AsyncStorage`, not in the cloud. There is no central "gate ownership" server.

Storage keys (KEYS)

```
wifigate:gates           # the gates array
wifigate:localProfile    # onboarding profile
                        (name/email/phone/provider/terms)
wifigate:phone           # phone number
wifigate:username        # display name
wifigate:language / :languagePreview
wifigate:gates:order     # display order
wifigate:gates:groups    # groups
wifigate:gates:dualControls # dual controls
wifigate:gates:toggleRelayStates # last relay state for TOGGLE gates
wifigate:firmware:url     # last firmware URL
wifigate:defaultCountryIso2 # default country code
wifigate:appleIdentityRecords # map userId-Apple identity (Apple doesn't send email every time)
wifigate:guestTable:*    # guest-table display preferences
```

Subtleties worth knowing

- ****loadGates()** normalizes on every load**: it merges `openHistory` with `lastOpenedAt`, normalizes `gateCommand`, `fotaChannel`, `hostEmail` (lowercase), and guest access.

- ****getJSON/setJSON swallow errors**** and return a fallback — a failed write

won't crash the app (but also won't alert — worth remembering while debugging).

- **Android limit:** `DEFAULT_ANDROID_ASYNC_STORAGE_LIMIT_BYTES = 6MB`.

`debugPrintAllLocalStorage()` prints used/remaining + a full dump — an excellent debug tool.

- **Cached profile:** `cachedLocalProfile` is loaded once;

`loadPhone/loadUsername/loadEmail` prefer it.

6. State Management

Source: `src/gatesStore.ts`.

This is a **hand-built observable store** — not Redux and not zustand (even though zustand is in the deps):

```
let _gates: Gate[] = [];
const listeners = new Set<() => void>();
export function subscribe(fn) { listeners.add(fn); return () =>
listeners.delete(fn); }
function notify() { listeners.forEach(f => f()); }
```

Every mutation (`addGate`, `updateGateRole`, `renameGate`, ...) works **immutably** (builds a new array), calls `saveGates`, `notify`, and when needed `syncGateWidgets()` + `syncAutoOpen()`.

initGates() — what happens at startup (very important)

22. `loadGates()` from `AsyncStorage`.

23. `normalizeGateRoleOnLoad` — migration of old guests (`isGuest:true` →

`role:"GUEST"`).

24. Comment migration (default).

25. ****Remove old gates without a `role`**** — the model is role-based only now.

26. `notify()`.

27. `cleanupExpired(true)` — **removes expired guest gates** and shows an Alert.

28. `syncGateWidgets()` + `syncAutoOpen()`.

Logs to look for:  `[Store] initGates()`,  `[Store] N gates loaded`,  `[Store] Removed N legacy gate(s)`,  `[Store] N guest gate(s) removed (expired)`.

7. The Core Flows

7.1 Onboarding / Sign in

Routes: `app/auth/login.tsx`, `app/auth/phone.tsx`.

The flow:

29. The user signs in through a provider (Google/Apple) → a normalized email is stored in the profile.
30. Profile name → country code → phone number → accept terms of use.
31. SMS OTP (Firebase Auth) → confirm code.
32. `saveLocalOnboardingProfile` stores everything and syncs phone/name to native Auto Open.

Why phone + email? In a secure BLE open, the firmware validates the inviter pair (host phone + host email) for guests. So the email is stored and passed in the command.

The gating itself is in `_layout.tsx` (`LayoutNavigator`): if not onboarded → redirect to `/auth/phone` (if there is already a provider identity) or `/auth/login`.

7.2 Adding a gate (QR / Manual)

- **Manual/QR** → `addGateFromQrOrManual` → role `PRE_ADMIN`.
- A QR that contains a compact security bundle (`dpk/psk`, without `protocol/keyId`) is normalized to `SECURE_V3` with `keyId="key-1"` (see `gateSecurity.ts`).
- The import/save path **rejects a gate without a security bundle** — new gates must have `SECURE_V3`.

7.3 Importing an invitation link (single/group/dual)

Sources: `linkService.ts`, `inviteLinkCrypto.ts`, `_layout.tsx`, `[app/gate/[id].tsx](../app/gate/[id].tsx)`. Full protocol in `README_WIFIGATE_LINK_INVITATIONS.md`.

The flow:

33. The OS opens `wifigate.io/wifigate-link/` (or `-api/`) or `wifigate://`.
34. `_layout.tsx` → `extractInvitationRoute` recognizes the type (single/group/dual/open-gate).
35. If not onboarded → the link is **queued** (`pendingInviteUrl`) until onboarding completes.
36. Route to `app/gate/[id].tsx` (or group/dual).
37. `parseIncomingLink` decrypts `ep=` (AES-GCM) into compact JSON.
38. **Validation**: valid decryption? time window? phone authorization (private vs public)?
39. `addGateFromGuestInvite` / `addGateFromUserInvite` / `addGate` (`PRE_ADMIN`) — merged into state per the merge rules.

Public vs Private (source: the links protocol):

- If field `p` (target phone) is present → the invitation is **private**: only the cached phone matching `p` can accept.
- If `p` is absent → **public**: restricted only by the time window (for guests).

Role-code mapping: `A`→ADMIN, `U`→USER, `G`→GUEST, `PU`→PRE_USER, `PA`→PRE_ADMIN. In practice: an admin invitation is sent as `r="PA"`, user as `r="PU"`, guest as `g=1`.

7.4 Opening a gate over BLE — the full flow

This is the most important flow. Orchestration in `openGateFlow.ts`:

```
runGateOpenFlow(gate)
├─ ensureBlePermissionsInteractive() → "permissions_blocked"
├─ ensureBluetoothReady() → "bluetooth_off"
├─ loadPhone() → "phone_missing"
├─ getGuestAccessValidation(gate) → "guest_expired"
├─ loads email/username/toggleRelayState
└─ openGateBLE({...}) → ble/openGate.ts
    └─ (handles onRoleUpgrade → updateGateRole)
        └─ saves toggleRelayState if gateCommand=TOGGLE
            └─ if OK and not PRE_* → markGateOpened (openHistory)
```

Status mapping: `OK`→ok, `UNAUTHORIZED`→unauthorized, `TIMEOUT`→timeout, `FAIL`→fail (and if the message indicates an active event/sensor → `event_active/sensor_inactive`).

Inside `openGateBLE` (in `ble/openGate.ts`):

40. **Scan / Connect** — `findConnectedGate`: scans devices, matches by

`systemId` (the advertised name), connects. There is a fast profile (`default`) and a slow one (`slow-android`). If there is a `preferredDeviceId` (from Auto Open) — it tries a direct connection first.

41. **Discover GATT** — requests MTU 517 (Android), discovers the **Nordic UART**

service:

- Service `6e400001-...`
- RX `6e400002-...` (write from app to gate)
- TX `6e400003-...` (notify from gate to app)
- **Inbox** — `createBleTextInbox`: assembles messages from chunks by `\n`,

supports both notify and polling (fallback), and filters local echo.

42. **Handshake SECURE_V3** — `client_hello` → `device_hello` → session keys (see Section 8).

43. **Encrypted command frame** — builds the command, adds the `authTag`, encrypts and sends (`seq=1`).

44. **Encrypted response frame** — reads, decrypts, returns a

`SecureDeviceResponse`.

7.5 Choosing the command by role and gate type

State	BLE command
USER/ADMIN, regular gate	<code>OPEN</code> (or <code>AUTO_OPEN</code>)
USER/ADMIN, TOGGLE gate	<code>TOGGLE_OPEN</code> / <code>TOGGLE_CLOSE</code> (by the stored <code>toggleRelayState</code>)
USER/ADMIN, PUSH_BUTTON gate	<code>PUSH_BUTTON_HOLD</code> (repeated) + <code>PUSH_BUTTON_RELEASE</code>
GUEST, regular gate	<code>OPEN_GUEST</code>
GUEST, TOGGLE gate	<code>TOGGLE_OPEN_GUEST</code> / <code>TOGGLE_CLOSE_GUEST</code>
PRE_USER	<code>ADD_USER</code> (confirms → USER)
PRE_ADMIN	<code>ADD_ADMIN</code> (confirms → ADMIN)

Guest open additionally passes `guestName`, `inviterName`, `inviterPhone`, `inviterEmail` — and the firmware validates the inviter pair before opening. If `inviterEmail` is missing in a guest invitation — the app stops with a message to request a new invitation.

Push button is a special case (source: `docs/ble-security-protocol-v3.md`): each HOLD frame refreshes the session TTL; the same BLE connection stays open while pressing; `seq` increments per frame; on finger-up a `PUSH_BUTTON_RELEASE` is sent. The firmware keeps a 1s watchdog as a fallback.

7.6 Sharing access (creating links)

Sources: `buildShareData` (single), `buildGroupShareData`, `buildDualControlShareData` in `linkService.ts`.

An admin can create: a guest link, a user link, an admin link, and a Nearby Wi-Fi setup link. The payload is built compact (`i,n,r,h,he,p,kid,dpk,psk...`), encrypted by `encryptInviteLinkPayload` into `ep=v1.<base64url>`, and combined into the public URL.

8. The Secure BLE Protocol

Sources: `src/ble/secureProtocol.ts` + the spec `docs/ble-security-protocol-v3.md`.

`SECURE_V3` is the **only supported secure protocol**. Both the app and the firmware must use v3.

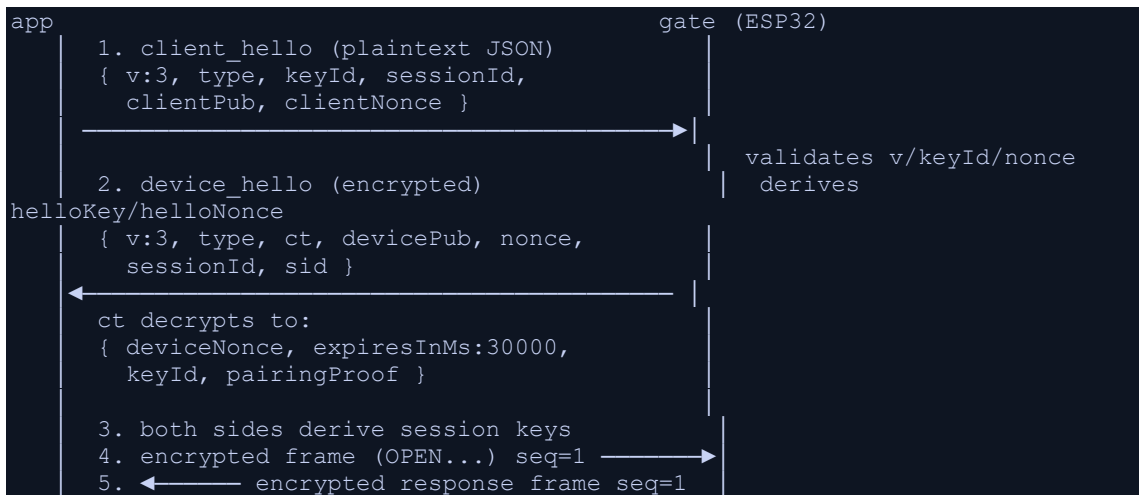
8.1 Provisioning bundle

Every secure gate holds a `Gate.security`:

```
{ protocol: "SECURE_V3",
  keyId: string,           // key slot, e.g. "key-1"
  devicePublicKey: string, // compressed P-256 public key, hex
  pairingSecret: string }  // 32-byte per-gate pairing secret, hex
```

8.2 The handshake — step by step

Constants (`secureProtocol.ts`): version `3`, default freshness `15000ms`, labels: `wifigate-ble-secure-v3` (transcript), `device-hello-proof-v3`, `command-auth-v3`.



8.3 Key derivation (key schedule)

```

ephemeral P-256 keypair (client)
sharedSecret = ECDH(clientSecret, devicePub).X    // 32 bytes (X of the
compressed point)

helloKey = HKDF-SHA256(sharedSecret, salt=clientNonce, info="hello-
key|sid|sessionId|keyId", 32)
helloNonce = HKDF-SHA256(sharedSecret, salt=clientNonce, info="hello-
nonce|...", 12)

// device_hello decrypts:
plaintext = AES-256-GCM(helloKey, helloNonce,
AAD="device_hello|systemId|sessionId").decrypt(ct)

// device identity check (proof-of-possession of pairingSecret):
pairingProof = HMAC-SHA256(pairingSecret, "device-hello-proof-
v3|systemId|session|keyId|clientPub|clientNonce|devicePub|deviceNonce")

transcriptHash = SHA256("wifigate-ble-secure-
v3|systemId|session|keyId|clientPub|clientNonce|devicePub|deviceNonce")

// session keys (salt = clientNonce || deviceNonce):
clientWriteKey = HKDF(sharedSecret, salt, "client-write-key|...", 32)
deviceWriteKey = HKDF(sharedSecret, salt, "device-write-key|...", 32)
clientNoncePrefix = HKDF(sharedSecret, salt, "client-nonce-prefix|...", 8)
deviceNoncePrefix = HKDF(sharedSecret, salt, "device-nonce-prefix|...", 8)
authKey = HKDF(pairingSecret, salt=transcriptHash, "command-auth-
key|...", 32)
  
```

8.4 Encrypted command frame

```

command = { cmd, systemId, timestamp, freshnessMs:15000, phone, role, name,
... }
commandHash = SHA256(stableJson(command))           // stable JSON
(sorted keys)
authTag = HMAC-SHA256(authKey, "command-auth-v3" || transcriptHash || seq(4B
BE) || commandHash)

innerPayload = { authTag, command }
nonce = clientNoncePrefix(8) || seq(4B BE)          // 12 bytes
ct = AES-256-GCM(clientWriteKey, nonce,
AAD="frame|c2d|systemId|session|seq").encrypt(innerPayload)
frame = { v:3, type:"frame", seq, ct }
  
```

The response (d2c) is symmetric: `deviceWriteKey`, `deviceNoncePrefix`, AAD with `d2c`. The firmware returns `{ type:"response", cmd, status:OK|FAIL|UNAUTHORIZED, gateCommand?, relayState?, ... }`.

stableJson matters: the command is serialized with alphabetically sorted keys and without `undefined` (`stableStringify`), so that the SHA256 is identical on both sides. Reordering fields would break the HMAC. Also, `systemId` is **omitted** from the wire command when it can be derived from the session.

8.5 Replay and Freshness resistance

Replay resistance comes from **multiple layers**: a fresh `sessionId`, fresh `clientNonce/deviceNonce`, per-frame `seq`, a transcript-bound `authTag`, session TTL, and freshness enforcement.

Freshness (firmware): rejects a command if `freshnessMs` is missing/0, if it exceeds the session TTL, or if the session age is greater than `freshnessMs`. The firmware logs the requested `freshnessMs`, the measured age, and the rejection reason.

8.6 Device key rotation (recovery)

If the gate returns a `devicePub` that differs from the stored one but the `pairingProof` is valid — the app accepts the new key (`usedStoredDevicePublicKey=false`) and refreshes it after the command succeeds. If decryption fails entirely → a friendly error: `"This gate's secure identity changed. Scan the latest WIFIGATE QR..."` (the device was probably reprovisioned/flash-formatted).

8.7 Transport parameters (BLE)

From `openGate.ts`: requested MTU `517` (attribute capped at 512B), chunk = `mtu-3`, message timeout `10000ms`, framing by `\n`, write with/without response depending on the characteristic. Special messages (`device_hello`, `frame`) are filtered by `systemId/sessionId/seq` — unexpected messages are skipped with `[BLE][SECURE] Ignoring unexpected...`

9. Invitation-Link Encryption

Source: `src/inviteLinkCrypto.ts`.

```
key = HKDF-SHA256(HMAC_URL_SECRET_bytes,
                  salt="wifigate-invite-link:key-salt",
                  info="wifigate-invite-link:aes-256-gcm", 32)
ciphertext = AES-256-GCM(key, managedNonce, AAD="wifigate-invite-link:v1").encrypt(JSON)
token = "v1." + base64url(nonce||ciphertext)
```

- `managedNonce(gcm)` — the nonce is randomized and prepended to the ciphertext automatically.
- The output is versioned `v1`. — allows a future version bump.
- **The website does not decrypt** — it just forwards the `ep=` to the app.

Security properties and an important limitation

✅ The payload is opaque in chats/browsers/previews. ✅ Tampering causes decryption failure in the app. ✅ The static website doesn't need to know the plaintext.

⚠️ **Limitation:** this is encryption with a **shared secret embedded in the build** — not end-to-end per recipient. Any trusted build that contains the URL secret can decrypt valid links. This is stronger than plain query params, but **not** equivalent to a server-issued/one-time/recipient-bound token.

10. Deep Linking

Source: `extractInvitationRoute` in `app/_layout.tsx`.

Supported schemes: `wifigate://` (native) and `https://wifigate.io` (paths: `wifigate-link`, `wifigate-api`).

The function parses a URL and decides on the target by priority order: `dual-control` → `group` → `open-gate` (widget) → `gate` (including `?ep=`, which becomes `gateId="invite"`). It also supports a nested `?link=`.

Subtlety: if the user is not onboarded, the link is queued (`pendingInviteUrl`) and processed only after onboarding completes (a `useEffect` on `isOnboarded`). Duplicates are filtered via `lastHandledInviteUrlRef`.

Logs to look for: `[Linking] event/initial URL received`, `[Linking] Routing invitation URL`, `[Linking] Queueing invitation URL until onboarding completes`, `[Linking] Ignoring unsupported URL`.

11. Auto Open

Sources: `src/auto-open/runtime.ts`, `config.ts`, `nativeAutoOpen.ts`, `autoOpenHeadlessTask.ts`.

The idea: when you approach a gate (RSSI strong enough), the app opens it automatically.

Important architecture: the shared JS runtime is actually a **fallback only**.

- On **iOS**, if there is a native module → `ensureNativeIosAutoOpenRunning` and the JS runtime is disabled.
- On **Android** → `ensureNativeAutoOpenServiceRunning` (a foreground service) and the JS runtime is disabled.
- The JS runtime (`runRuntimeTick`) runs only if there is no native — it scans candidates (`scanAutoOpenCandidatesBLE`), picks the strongest, and dispatches to `processAutoOpenTrigger`.

Why native? Because Auto Open must work in the background — JS in React Native does not run reliably in the background, certainly not for continuous BLE scanning.

`startAutoOpenRuntime()` is called once from `_layout.tsx` (a `useEffect`).

Logs: `[AutoOpen][Runtime]`, `[AutoOpen][iOS]`, `[AutoOpen][Android]`, `[BLE][AUTO_OPEN]`
`Candidate scan matched N gate(s)`.

12. FOTA / OTA

Sources: `src/fota/publicFota.ts` (Firebase, the new one), `driveFota.ts` (Google Drive, legacy), `app/firmware-update.tsx`. Guide: `README_OTA_FIREBASE.md`.

The firmware is read from **Firestore Storage** by two dimensions:

- QR channel: `fotaChannel = dev / prod`.
- hardware variant: `8MB / 16MB`.

⇒ 4 feeds: `{dev|prod}-fota/{8mb|16mb}/manifest.json`. The app downloads encrypted `.wgfota` chunks listed in the manifest and validates `flashVariantMb` before continuing.

The gate-side flow: `OTA_OPEN` over BLE → the gate returns the AP SSID/password/token → the app connects and streams the firmware. The public config is in `app.json` (`expo.extra.fotaStorage*`).

Logs: `[FOTA]`.

13. Nearby Wi-Fi Setup

Sources: `src/components/gate-settings/admin-access/NearbyWIFIGATELink.tsx`, `app/wifigate-setup.tsx`.

Lets an admin open the gate's local AP and enter the setup page:

45. The admin chooses "Nearby WIFIGATE Link".
46. Request BLE permissions + confirm Bluetooth.
47. Send `WIFI_OPEN` over BLE.
48. If the gate approves → it returns the current SSID/password.
49. Scan nearby Wi-Fi networks (`react-native-wifi-reborn`).
50. Connect to the gate's AP.
51. Open `http://192.168.4.1/` in a WebView.

The WebView is hardened: restricted to the gate's local host, blocks arbitrary external navigation, blocks app-scheme navigation, supports CSV download, and offers reload/recovery.

⚠ The BLE protocol does **not** secure this HTTP page — it is a separate local-network workflow.

Logs: [NearbyWIFIGATELink], [WebView], [WiFi].

14. Important Logs

`logger.ts` defines `log/warn` that run ****only in `__DEV__`****. But note: many of the critical logs (BLE, Store, Linking) use `console.log/console.warn` **directly**, meaning they appear in regular builds too. So the BLE trace is the main field debug tool.

14.1 Tag catalog (by frequency)

Tag	Where	What it tells you
[BLE]	ble/openGate.ts	scan, connect, MTU, TX/RX chunks, timeouts
[BLE] [SECURE]	ble/openGate.ts	handshake, client/device hello, encrypted frames
[BLE] [AUTO_OPEN]	ble/openGate.ts	candidate scan for Auto Open
[Store]	gatesStore.ts	load, merge, downgrade prevention, cleanup
[GateOpenFlow]	openGateFlow.ts	the orchestration steps of the open
[MyGates]	features/my-gates	home screen (collections, tiles)
[GateScreen]	app/gate	gate details screen
[AutoOpen] *	auto-open/*	runtime, iOS, Android, UI
[Linking]	_layout.tsx	receiving deep links and routing
[ENCRYPTED_INVITE_PARAM]	linkService	parsing <code>ep=</code>
[InviteLinkCrypto]	inviteLinkCrypto.ts	payload decryption failure
[FOTA]	fota/*	OTA
[NearbyWIFIGATELink] [WebView] [WiFi]	setup flow	Nearby Wi-Fi
[GateWidget] [WidgetHeadlessOpen] [WidgetOpenGate]	widgets/*	widget open
[storage]	storage.ts	usage, dump, write failures
[Contacts] [Camera]	permissions	permissions

14.2 What to look for per flow (debug playbook)

A gate open fails:

```
[GateOpenFlow] Checking BLE permissions / Bluetooth state
[BLE] Scanning for gate for Nms...
[BLE] Gate not found. Move closer...      ← TIMEOUT (proximity / advertised name)
[BLE] Requested MTU 517, negotiated N
[BLE][SECURE] Sending client_hello. sessionId=...
[BLE][SECURE] Waiting for device_hello...
[BLE][SECURE][DEVICE_HELLO_PARSE] ...    ← bundle mismatch (reprovision?)
[BLE][SECURE] Sending encrypted OPEN frame. seq=1
[GateOpenFlow] BLE result: { status: ... }
```

Gate identity changed: look for `Could not decrypt device_hello with the stored secure bundle / pairing proof mismatch` → need to re-scan the QR.

A link import fails: `[Linking] Routing invitation URL → [InviteLinkCrypto] Failed to decrypt invitation payload` (tampering/bad secret) or an Alert "Invalid invitation / Access expired / Unauthorized".

A guest disappeared: `[Store] N guest gate(s) removed (expired)` — expired at startup.

A downgrade was prevented: `[Store] Preventing role downgrade for gate X: ADMIN stays`.

15. Permissions and Platform

Configured in `app.json` and native files. The app needs:

- **Camera** — QR scanning.
- **Bluetooth** — gate communication.
- **Location (Android)** — historically required for BLE/Wi-Fi scanning.
- **Nearby Wi-Fi devices (Android)**.
- **Local network (iOS)** — access to `192.168.4.1`.

Deep linking is configured for `wifigate://` and `https://wifigate.io`.

- Permissions entry point:

```
src/permissions/appLaunch.ts (checkAppLaunchPermissions = BLE + Bluetooth ready;  
prepareStartupPermissionsForSignIn = contacts + BLE).
```

- Bluetooth state: `ble/bluetoothState.ts` (`ensureBluetoothReady`).
 - BLE permissions: `ble/permissions.ts` (`ensureBlePermissionsInteractive`,
`hasBlePermissions`).
-

16. Secrets and Build Inputs

Required inputs (some are **gitignored**):

- `src/crypto/HMAC_SECRETS.json` — a **secret**. Used to reconstruct `HMAC_URL_SECRET` (link encryption) and any legacy URL/HMAC protection. Derived through `src/crypto/hmacSecrets.ts`.
- `GoogleService-Info.plist` (iOS) + `google-services.json` (Android) — Firebase.

EAS: the script `scripts/eas-pre-install.js` places `HMAC_SECRETS.json` into the build environment (`eas-build-pre-install`). Outside the repo, provided via an environment variable/CI.

⚠ If `HMAC_URL_SECRET` is not configured — `deriveInviteLinkKey` throws, and all link encryption/decryption fails.

17. Local Development and Release

From package.json:

```
npm install
# provide src/crypto/HMAC_SECRETS.json (or HMAC_SECRETS_JSON in EAS)
# make sure Firebase files exist for the environment

npm run start:dev-client    # expo start --dev-client
npm run start:usb           # Android over USB (PowerShell script)
npm run reset:app           # reset Android app state
npm run reset:usb           # reset + relaunch

npm run lint                # eslint
npm run typecheck           # tsc --noEmit
npm run verify              # lint + typecheck ★ run before shipping
```

Release checklist (device smoke test)

- ☐ Sign-in via phone OTP
- ☐ QR / manual import
- ☐ Encrypted link import
- ☐ Guest invite flow
- ☐ User/admin invite flow
- ☐ Gate open over BLE
- ☐ Nearby Wi-Fi setup
- ☐ CSV export from the setup page (if relevant)

Build and submit with EAS after the smoke test.

18. Security Boundaries

It is important to understand **what is not protected**:

- `SECURE_V3` protects the BLE transport **only** — not the HTTP setup page (`192.168.4.1`), not link sharing, and not credential storage on the phone.
 - Invitation links are encrypted and tamper-resistant, but use a ****shared secret** in the build** — not a server-issued/one-time/recipient-bound token.
 - Gate state is **local-first** in AsyncStorage — there is no cloud-synced gate ownership.
 - Roles are enforced by both app logic and device behavior, but the ****ultimate** hardware authorization must be enforced in the firmware** — do not trust the UI.
-

19. Cheat Sheet

Files by task

Want to understand...	Read
BLE crypto and message flow	docs/ble-security-protocol-v3.md, secureProtocol.ts
BLE core (scan/connect/frames)	ble/openGate.ts
open orchestration	openGateFlow.ts
role and merge rules	models.ts, gatesStore.ts
creating/importing links	linkService.ts, inviteLinkCrypto.ts
full links protocol	README_WIFIGATE_LINK_INVITATIONS.md
deep linking	app/_layout.tsx
Nearby setup	NearbyWIFIGATELink.tsx , app/wifigate-setup.tsx
OTA	README_OTA_FIREBASE.md, fota/publicFota.ts
storage	storage.ts

BLE commands (SecureCommandName)

```
OPEN    TOGGLE_OPEN  TOGGLE_CLOSE  AUTO_OPEN
OPEN_GUEST  TOGGLE_OPEN_GUEST  TOGGLE_CLOSE_GUEST
PUSH_BUTTON_HOLD  PUSH_BUTTON_RELEASE
ADD_USER  ADD_ADMIN
WIFI_OPEN  OTA_OPEN  GET_FW_VERSION
GET_AUTO_OPEN_STATUS  CHECK_AUTO_OPEN_RSSI  GET_GATE_COMMAND
```

gateCommand values in the response

1=PULSE 2=PULSE_AFTER_PULSE 3=PULSE_AFTER_PULSE_AFTER_PULSE 4=TOGGLE
5=PUSH_BUTTON

Nordic UART UUIDs

```
Service      6e400001-b5a3-f393-e0a9-e50e24dcca9a
RX (write)   6e400002-b5a3-f393-e0a9-e50e24dcca9a
TX (notify)  6e400003-b5a3-f393-e0a9-e50e24dcca9a
```

This document was written by reading the source code. When something in the code changes (especially [secureProtocol.ts](#), [linkService.ts](#), [gatesStore.ts](#)) — update this too.